



Universidade do Minho
Licenciatura em Engenharia Informática

Unidade Curricular de Laboratórios de Informática III

Relatório Fase II

Ano Lectivo de 2025/2026

Gonçalo Fernandes da Silva e Sousa
A110977 - goncalofssousa

Pedro Tiago Moniz Morais
A109719 - tiagomorais121

Tiago da Cunha Pereira
A112010 - tiagocpereira13

Índice

1. Introdução	3
2. Sistema	4
2.1. Arquitetura definida	5
3. Discussão	6
4. Conclusão	7

Introdução

O trabalho prático teve como objetivo o desenvolvimento de um programa capaz de processar, validar e consultar conjuntos de ficheiros fornecidos em formato CSV. Este projeto insere-se num contexto de verificação de informação, onde é fundamental garantir a correção dos dados de entrada, eficiência e tempo de execução do programa.

O sistema foi concebido para ler os diferentes datasets disponibilizados e realizar uma validação rigorosa dos mesmos. Os resultados obtidos foram posteriormente comparados com os ficheiros de saída esperados, permitindo testar e verificar o comportamento do programa de forma automática e controlada. Ainda assim, tivemos de criar e executar um conjunto de queries e posteriormente efetuar um programa-testes para a verificação das mesmas.

Entre as principais características do projeto destaca-se a implementação de hashtables, que desempenhou um papel essencial na organização e gestão eficiente dos dados. Esta estrutura permitiu associar de forma direta e rápida cada elemento às respetivas *keys*, facilitando o acesso e a manipulação da informação ao longo do programa. A biblioteca GLib revelou-se fundamental, ao fornecer um conjunto de estruturas de dados e funções utilitárias que contribuíram para um desenvolvimento mais eficiente e seguro. Por fim, a organização do código foi também uma prioridade, sendo os diferentes componentes distribuídos por múltiplos ficheiros `.c` e `.h`. Os ficheiros `.h` foram utilizados para declarar as estruturas de dados e as funções, permitindo separar as definições das funções da sua implementação real, garantindo assim um código mais estruturado e de fácil manutenção.

Sistema

O sistema foi concebido segundo uma **arquitetura modular**, sendo dividido em vários componentes, cada um responsável por uma área específica de funcionalidade. Esta abordagem promove uma organização clara do código, facilita a manutenção e evolução do sistema e permite uma compreensão mais direta das responsabilidades associadas a cada módulo.

A estrutura central do programa assenta em módulos dedicados às principais entidades do sistema, nomeadamente **aircraft**, **airport**, **flight**, **passenger** e **reservation**. Cada entidade é implementada num ficheiro .c, onde se encontra definida a estrutura de dados utilizada para representar as suas instâncias, bem como os respectivos *getters* e *setters* que permitem o acesso controlado e a atualização dos seus campos. Estas estruturas são **opacas**, sendo o seu acesso direto restrito ao módulo onde são definidas, garantindo o encapsulamento dos dados. Cada módulo disponibiliza ainda funções responsáveis pela alocação e libertação de memória das instâncias da entidade. Nos respetivos *headers* (.h) encontram-se apenas as declarações das funções públicas e os tipos necessários à interação com outros módulos, assegurando uma separação clara entre **interface** e **implementação**.

Associado a cada entidade existe um **manager**, responsável por definir e gerir as estruturas de dados onde as instâncias são armazenadas, bem como por fornecer operações de inserção e gestão dessas instâncias. Para além disso, alguns managers integram estruturas auxiliares utilizadas exclusivamente no processamento de determinadas queries, tais como `rangeArgs` e `ArrayAirports` no manager de **airports** (Query 3), `flightDelay` no manager de **flights** (Query 5), `nacionalidade` no manager de **passengers** (Query 6) e `weekData` no manager de **reservations** (Query 4). Estas estruturas foram introduzidas com o objetivo de armazenar dados intermédios e cálculos parciais, nomeadamente relativos a intervalos temporais e estatísticas, evitando o armazenamento redundante de informação e reduzindo o consumo de memória.

A coordenação entre os diferentes managers é assegurada pelo **ManagerGlobal**, que agrega todas as estruturas de gestão do sistema e funciona como um ponto central de acesso às mesmas. Este componente permite que funções de carácter genérico, como as responsáveis pelo *parsing* de dados ou pela interpretação e execução de queries, operem sobre uma única referência, em vez de dependerem de múltiplos managers individuais. Esta abordagem simplifica a passagem de argumentos e contribui para uma arquitetura mais limpa, modular e fácil de manter.

Para o carregamento dos dados, o sistema integra um componente dedicado à **leitura, validação e processamento dos datasets**, organizado em diferentes etapas que refletem o fluxo de processamento da informação. O módulo `dataset_reader` é responsável por validar o caminho fornecido para os ficheiros de dados e identificar o tipo de ficheiro a processar (i.e, a que entidade corresponde), atribuindo posteriormente a sua leitura ao módulo de *parsing*. O módulo `dataset_parser` procede à leitura efetiva dos ficheiros, linha a

linha, dividindo cada linha em tokens/colunas.. Estes tokens são então passados às funções de *load* específicas de cada entidade. As funções de load, específicas de cada entidade, são responsáveis por efetuar a validação semântica dos tokens obtidos durante o parsing. Caso os dados sejam considerados válidos, é alocada uma nova instância da entidade, cujos campos são inicializados com base nos tokens processados. De seguida, esta instância é inserida nas estruturas de dados apropriadas através do respectivo manager. Adicionalmente, estas funções podem interagir com outros managers, de forma a armazenar informação auxiliar necessária para a execução de determinadas queries, evitando assim o armazenamento redundante de dados.

De forma resumida, o fluxo de carregamento dos dados pode ser descrito como:

validação do caminho → leitura do ficheiro → divisão das linhas em tokens → validação dos dados → criação da instância → inserção nas estruturas dos managers.

O sistema inclui também uma componente responsável pelo **tratamento de input e output**, que gere a interpretação, execução e produção dos resultados das queries. Este fluxo inicia-se no **Interpreter**, que recebe uma linha de input e a encaminha para o módulo de *parsing* de queries (*query_parse_line*). Este módulo analisa a linha, identificando o número da query, os respetivos argumentos e o separador utilizador para o output, produzindo uma estrutura auxiliar Query que encapsula toda a informação necessária à sua execução. De seguida, o Interpreter invoca o módulo *execute_query*, que executa a query correspondente através do módulo específico associado a cada tipo de query. Cada query produz um objeto de output, sendo o módulo de output responsável pela criação do ficheiro de resultados e pela escrita da informação produzida, ou então no caso do programa interativo de fazer print do resultado obtido.

Este *Interpreter* pode ser utilizado tanto pelo **programa principal** como pelo **modo interativo** do sistema. As funções relativas ao funcionamento do modo interativo (exceto a *main*), estão implementadas no módulo *read_input_interactive*. O utilizador começa por introduzir o caminho para os ficheiros do dataset, que é validado e processado através do fluxo de carregamento descrito anteriormente. Após o *load* dos dados, o programa permite ao utilizador introduzir queries de forma iterativa, validando o tipo de query e os respetivos argumentos e, sempre que estes sejam considerados válidos, encaminhando-os para o *Interpreter* para execução.

Esta integração entre módulos garante um fluxo de execução bem definido, uma clara separação de responsabilidades e uma arquitetura extensível e eficiente, adequada à evolução futura do sistema.

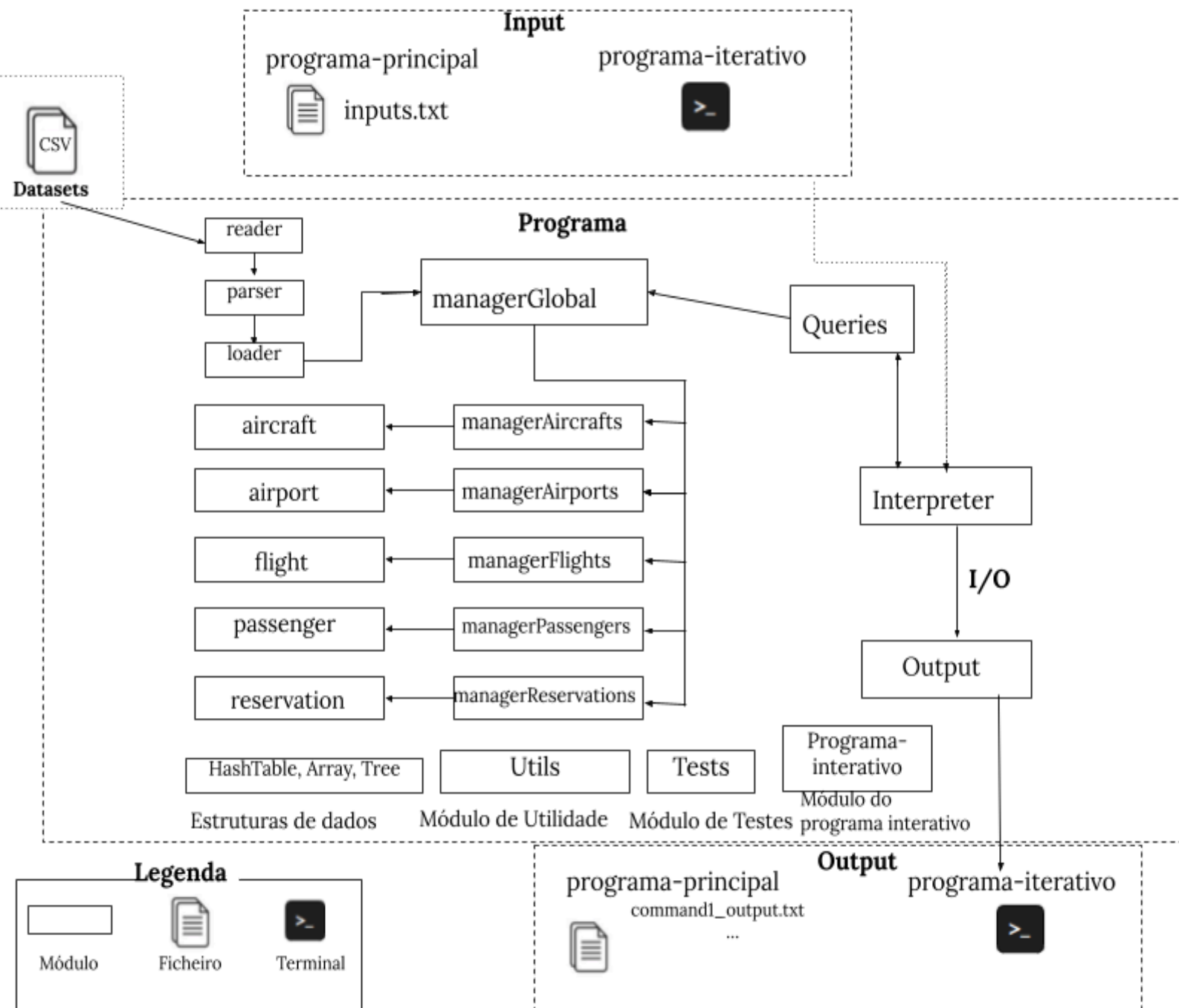


Figura 1: Arquitetura para a aplicação desenvolvida.

Discussão

No desenvolvimento do programa, foram aplicadas técnicas de **encapsulamento** para controlar o acesso aos dados internos dos módulos. As **estruturas opacas** foram utilizadas para esconder a implementação interna das entidades do sistema, permitindo que apenas as funções disponibilizadas sejam usadas para interagir com elas. Para aceder a propriedades específicas sem expor diretamente os dados, foram implementadas funções **getters**, garantindo que qualquer manipulação ou leitura siga regras definidas pelo módulo e evitando alterações indevidas.

Na **query 1**, a eficiência é assegurada através do uso de uma **hash table** no *ManagerAirports*, que armazena os aeroportos indexados pelo seu código. Esta estrutura permite obter diretamente a informação de um aeroporto específico em tempo praticamente constante, sem necessidade de percorrer conjuntos de dados adicionais. A query limita-se a uma operação de pesquisa direta, recorrendo exclusivamente às funções do manager para aceder aos dados da entidade Airport. Desta forma, a escolha da hash table como estrutura principal garante uma resposta imediata à query, mesmo com um elevado número de aeroportos, mantendo simultaneamente encapsulamento e integridade da informação.

Para a **query 2**, o *ManagerAircrafts* utiliza uma combinação de **hash tables** e **arrays dinâmicos** para garantir eficiência nas operações e nas consultas. As aeronaves são armazenadas numa hash table indexada pelo seu identificador, permitindo inserções e pesquisas em tempo praticamente constante, para por exemplo pode ser a aircraft de um flight existe em tempo constante. Para suportar a ordenação e o acesso sequencial aos resultados, é mantido um GArray com todas as aeronaves, bem como uma hash table adicional que agrupa aeronaves por fabricante, cada uma associada a um array próprio. Esta separação evita percorrer todo o conjunto de dados quando a query é filtrada por fabricante, reduzindo significativamente o custo da operação. Os arrays são ordenados uma única vez com base no número de voos, permitindo que a query aceda diretamente aos resultados já ordenados. Desta forma, a escolha destas estruturas de dados assegura uma execução eficiente da query,, sem duplicação de informação.

Para a **query 3**, a eficiência é garantida através da combinação entre estruturas persistentes e uma **estrutura auxiliar baseada em arrays**. Os aeroportos são geridos pelo *ManagerAirports*, que mantém a informação principal indexada por estruturas de acesso rápido, enquanto os voos válidos são organizados temporalmente no momento da inserção. Para o processamento da query, é criada a estrutura auxiliar *ArrayAirports*, um array indexado onde cada posição corresponde a um aeroporto. Esta estrutura permite acumular, em tempo linear, o número de voos por aeroporto dentro do intervalo de datas pedido, evitando percursos repetidos sobre os dados originais. Durante a agregação, o máximo é atualizado incrementalmente, eliminando a necessidade de uma passagem extra para determinar o aeroporto com mais voos. Desta forma, a utilização de arrays indexados e estruturas auxiliares temporárias permite processar a query de forma mais eficiente, mesmo

para intervalos de datas extensos, sem alterar as estruturas principais nem introduzir redundância de armazenamento.

No caso da **query 4**, o *ManagerReservations* utiliza uma **GTree**, onde cada nó corresponde a uma semana identificada por uma data inteira, que corresponde ao domingo da mesma. Estas contêm uma estrutura auxiliar *weekData*, que agrega informação específica dessa semana, nomeadamente uma hash table *passengerTotals*, responsável por armazenar o total acumulado (por exemplo, valor gasto ou número de reservas) por passageiro. Adicionalmente, cada *weekData* mantém um *GArray* com o **top 10 de passageiros** dessa semana, bem como uma flag *dirty* que evita recomputações desnecessárias. O *manager* guarda ainda apontadores para a última semana consultada, permitindo iteração eficiente entre semanas consecutivas, e mantém informação agregada global (*all time*), calculada a partir dos tops semanais sem alterar a estrutura principal.

Por sua vez, na **query 5**, é utilizada a entidade auxiliar *FlightDelay*, que permite acumular dados intermédios sobre atrasos por companhia aérea. Esta estrutura armazena o atraso total, o número de voos e o atraso médio calculado, evitando redundância de armazenamento. O *ManagerFlights* mantém uma hashtable *airlineDelays* que associa cada companhia aérea à sua respectiva estrutura *FlightDelay*, bem como um *GArray* com as companhias ordenadas por atraso médio. A ordenação é realizada apenas uma vez, após o cálculo dos valores médios, garantindo eficiência na execução da query.

Relativamente à **query 6**, o *ManagerPassengers* recorre a uma hash table principal para armazenar os passageiros indexados pelo número de documento, e a uma estrutura auxiliar *nationality* para agrupar informação por nacionalidade. Cada entrada desta estrutura corresponde a uma nacionalidade, que contém uma hash table *airportsCount*, usada para contabilizar o número de passageiros dessa nacionalidade por aeroporto de destino. Esta estrutura mantém também o aeroporto com maior número de passageiros (*code_max*) e o respetivo valor máximo, permitindo responder à query de forma eficiente, sem necessidade de percorrer novamente todos os voos ou passageiros.

Em todos os casos, os dados **não são acessados diretamente**; qualquer leitura ou manipulação é feita através de **funções públicas dos managers**, mantendo a integridade e consistência do sistema.

Essa combinação de **encapsulamento, estruturas opacas, getters, hashtables e estruturas auxiliares específicas para cada query** garante que o sistema seja **eficiente, modular e seguro**, permitindo processar consultas complexas de forma rápida, mesmo em grandes volumes de dados, sem comprometer a organização ou integridade das estruturas principais.

Para reduzir os custos de memória, visto que não era necessário armazenar informação sobre as reservas para depois processar nas queries, decidimos não alocar instâncias desta entidade, nem criar uma estrutura de dados específica, como hashtable para consultar as mesmas por ID. Tal como neste caso decidimos não armazenar campos desnecessários em

algumas entidades como as datas sem ser o departure nos Flights, a latitude, longitude e ICAO dos airports e ainda o género, telemóvel, endereço e foto nos passengers.

O **programa-testes** foi desenvolvido para validar automaticamente os resultados das queries implementadas, comparando os ficheiros de saída gerados pelo programa com os outputs esperados. Para cada query, o programa verifica se todas as linhas coincidem, identifica a primeira linha onde ocorre uma discrepância e regista o número total de testes realizados, assim como os testes que passaram com sucesso. Adicionalmente, o programa mede o tempo de execução de cada query e o tempo total, permitindo uma análise de desempenho detalhada.

Nos testes realizados com o **dataset2 sem_erro**s, todas as queries obtiveram 100% de sucesso, em todas as máquinas:

Q1: 50 de 50 testes; **Q2:** 50 de 50 testes; **Q3:** 50 de 50 testes;

Q4: 50 de 50 testes; **Q5:** 50 de 50 testes; **Q6:** 50 de 50 testes;

Memória Utilizada: 2299MB

Nos testes realizados com o **dataset com_erro**s, todas as queries obtiveram 100% de sucesso, em todas as máquinas:

Q1: 50 de 50 testes; **Q2:** 50 de 50 testes; **Q3:** 50 de 50 testes;

Q4: 50 de 50 testes; **Q5:** 50 de 50 testes; **Q6:** 50 de 50 testes;

Memória Utilizada: 2299MB

Máquina	Dataset2 sem_erro	Dataset2 com_erro
Linux	31s	34s

Figura 2: Tabela com o tempo total nas 2 máquinas usadas (2 Linux).

Os resultados obtidos pelo programa-testes mostram um desempenho consistente e confiável em todos os datasets utilizados. No dataset2 sem erros, todas as queries apresentaram 100% de sucesso, o que indica que a implementação das funções e das estruturas de dados está correta para entradas válidas. Este resultado demonstra que a lógica de processamento, a utilização das hashtables e a ordenação das listas ligadas funcionam como esperado, permitindo acesso rápido aos dados e geração correta dos outputs.

No dataset2 com erros, os resultados também foram consistentes, apesar da presença de entradas inválidas. O programa conseguiu identificar corretamente as linhas problemáticas, registá-las quando necessário e continuar o processamento das restantes entradas. O facto

de todos os testes terem produzido os outputs esperados confirma que as técnicas de validação e tratamento de erros foram implementadas de forma eficaz.

Em termos de desempenho, verificou-se que o dataset sem erros foi processado em cerca de 33 segundos, enquanto o dataset com erros demorou aproximadamente 43 segundos. Esta diferença está diretamente relacionada com o tempo adicional gasto na verificação de erros e no registo das linhas inválidas, o que é esperado em qualquer sistema que realiza validação de dados.

Relativamente à memória utilizada, o dataset sem erros consumiu cerca de 2299 MB, enquanto o dataset com erros consumiu ligeiramente mais, cerca de 2286 MB.

Conclusão

O trabalho desenvolvido foi bem sucedido e refletiu o empenho de todo o grupo. Tivemos de trabalhar com hashtables pela primeira vez, o que constituiu um desafio inicial, mas a experiência correu muito bem e conseguimos aplicar esta estrutura de forma eficaz conseguindo assim aplicar e aprender esta nova estrutura de dados para nós.

Além disso, aprendemos a aplicar encapsulamento, garantindo que os dados internos das estruturas fossem protegidos e manipulados apenas através de funções controladas. Além disso, o sistema foi desenvolvido de forma modular, com cada entidade e manager organizados em módulos distintos, algo que não tínhamos aprofundado anteriormente.

No geral, o grupo considera que a fase 2 do projeto foi uma experiência muito positiva, permitindo não só consolidar conhecimentos já adquiridos, mas também explorar novas técnicas de forma prática e bem sucedida.

